

Praktikum Pemrograman Berbasis Objek

Pertemuan 7 — Exception Handling

Outline Materi Pertemuan 7

Error & Exception

- Definisi dan perbedaan Error vs Exception
- Konsep Exception Handling
- Alur normal vs alur penanganan exception

Hierarki Exception di Java

- Class `Throwable`
- Checked Exception vs Unchecked Exception
- Kondisi kemunculan masing-masing exception

Melempar Exception

- Penggunaan kata kunci `throw`
- Kondisi ideal untuk melempar exception secara eksplisit

Menangani Exception

- `try` — blok eksekusi normal
- `catch` — blok penanganan exception
- `finally` — blok eksekusi akhir

Exception Propagation

- Penggunaan kata kunci `throws`
- Alur propagasi exception antar metode

Custom Exception

- Implementasi class exception kustom
- Studi kasus: Exception pada REST API

Error & Exception

Memahami Definisi dan Perbedaannya

Error vs Exception

Error

Kondisi kritis di mana program tidak dapat berjalan semestinya. Error umumnya dipicu oleh:

- Kesalahan sintaks
- Kesalahan logika fundamental
- Masukan yang tidak valid
- Faktor eksternal sistem (jaringan, memori penuh)

Exception

Mekanisme pada tingkat runtime yang dirancang untuk mengalihkan alur eksekusi dari alur normal menuju alur penanganan khusus ketika terjadi anomali.

Alur Eksekusi dalam Konteks Exception

Catatan Penting: Error TIDAK SAMA DENGAN Exception

Error adalah kejadian dari adanya kesalahan pada sebuah program. Exception adalah cara komputer "membuat rangkuman informasi" mengenai error agar bisa "ditangani" oleh penulis program. Exception memerlukan penanganan khusus dan tidak menjalankan alur normal sebuah program.

Alur Normal

Alur eksekusi program utama yang berjalan sesuai dengan logika bisnis yang diharapkan dalam kondisi ideal.

Alur Penanganan Khusus

Blok kode yang spesifik didefinisikan untuk merespons dan menangani suatu exception agar program terhindar dari penghentian paksa (crash) dan memberikan informasi mengenai error menjadi lebih rapi dan informatif.

Konsep Exception Handling

Mekanisme penanganan khusus tersebut di dalam pemrograman disebut sebagai exception handling.

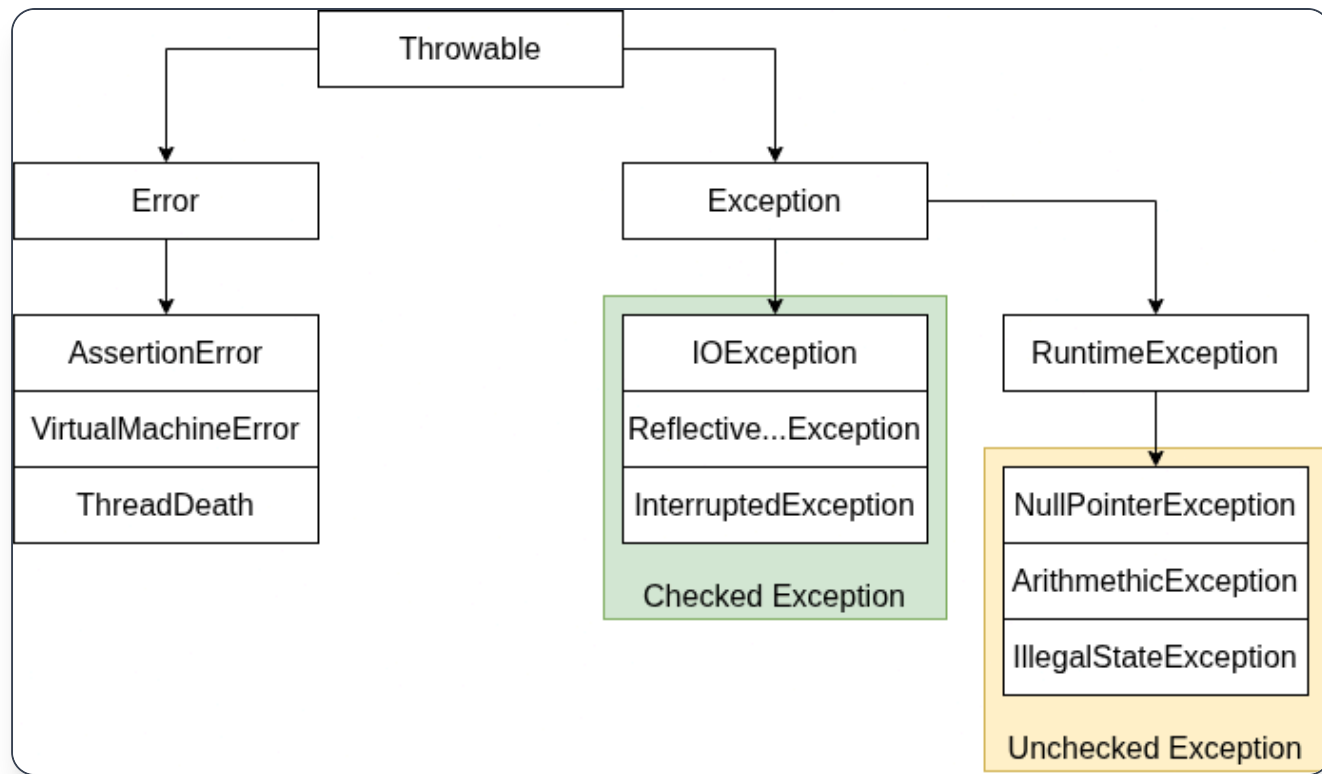
Analogi Konseptual

Exception handling dapat dianalogikan seperti prosedur darurat pada penerbangan. Alur normal merepresentasikan penerbangan yang berjalan mulus. Exception merepresentasikan kendala teknis pada sistem pesawat. Exception Handling merupakan prosedur darurat standar yang diterapkan oleh kru untuk mengatasi kendala tanpa mengakibatkan kegagalan penerbangan.

Hierarki Exception di Java

Memahami Struktur Class Exception

Hierarki Exception di Java



Class Throwable & Jenis Exception

Class Throwable

Java memiliki class `Throwable`, yang artinya seluruh object dari class `Throwable` atau turunannya dapat "dilempar" menggunakan keyword `throw`.

Keyword `throw` digunakan untuk melempar sinyal bahwa ada **exception** / kesalahan ketika runtime.

Dua Jenis Exception

Java memiliki dua jenis exception yang lumrah dibahas:

- **Checked Exception**
- **Unchecked Exception**

Keduanya terjadi di level runtime. Perbedaannya adalah bagaimana Java memperlakukan keduanya.

Contoh: melempar exception secara eksplisit

```
throw new RuntimeException("Ada yang salah!");
```

Baik exception yang muncul karena *error* maupun yang *sengaja dilempar*, keduanya tetap perlu penanganan khusus.

Checked vs Unchecked Exception

Checked Exception

Kondisi ini diidentifikasi saat proses kompilasi. Kompilator mewajibkan pengembang untuk menyediakan penanganan (try-catch) atau mendeklarasikannya (throws) agar kode dapat dikompilasi. Karena kemungkinan kesalahan sudah diidentifikasi sejak awal, mitigasi sangat diwajibkan.

CONTOH SKENARIO:

- Kegagalan koneksi ke database
- Operasi baca/tulis pada berkas yang tidak ditemukan
- Kegagalan pada transmisi jaringan

Unchecked Exception

Kondisi ini tidak wajib ditangani saat kompilasi. Umumnya disebabkan oleh kesalahan logika pemrograman yang baru terdeteksi pada saat eksekusi (runtime) dan kadang tidak disadari sejak awal.

CONTOH SKENARIO:

- Pembagian bilangan bulat dengan nol
(`ArithmeticException`)
- Kesalahan konversi tipe data
(`NumberFormatException`)
- Akses referensi objek bernilai null
(`NullPointerException`)

Melempar Exception

Keyword `throw` & Penggunaannya

Melempar Exception Secara Eksplisit

Exception dapat dipicu secara sengaja oleh pengembang melalui kode, meskipun tidak terjadi kesalahan sistem secara teknis.

Tujuan Pelemparan Exception Eksplisit

Digunakan untuk mengendalikan alur eksekusi spesifik dalam merespons ketidaksesuaian aturan bisnis (business logic), antara lain:

- Validasi ketersediaan entitas data pada sistem
- Validasi integritas dan batasan nilai parameter masukan
- Pengecekan otoritas dan otentikasi hak akses

Catatan Penting:

Setiap exception, baik yang timbul akibat kegagalan sistem maupun yang dilempar secara sengaja, memerlukan mekanisme penanganan untuk menjaga stabilitas program.

Contoh Kode Pelemparan Exception

Menggunakan kata kunci `throw` untuk memicu exception ketika aturan bisnis tidak terpenuhi.

```
public class BankAccount {  
    private double balance = 500.0;  
  
    public void withdraw(double amount) {  
        // Validasi aturan bisnis  
        if (amount > balance) {  
            // Melempar exception jika kondisi gagal terpenuhi  
            throw new IllegalArgumentException(  
                "Saldo tidak mencukupi untuk memproses transaksi penarikan ini."  
            );  
        }  
  
        // Alur normal jika validasi berhasil  
        balance -= amount;  
    }  
}
```

`try` — `catch` — `finally`

Metode Penanganan Exception Paling Umum

Struktur `try` — `catch` — `finally`

Struktur sintaksis sentral yang diadaptasi oleh berbagai bahasa pemrograman modern untuk mengelola eksekusi exception.

Blok `try`

Ruang lingkup untuk menempatkan kode program yang mengeksekusi operasi normal sesuai spesifikasi algoritma.

Apabila terjadi exception di dalam blok ini, eksekusi baris selanjutnya akan dihentikan dan dialihkan seketika ke blok penanganan.

Blok `catch`

Ruang lingkup yang dieksekusi secara spesifik untuk merespons dan menangkap exception yang dideklarasikan.

Dalam pengembangan API, blok ini umumnya dimanfaatkan untuk menerjemahkan kegagalan sistem menjadi struktur objek `respons error`.

Blok `finally` (opsional)

Ruang lingkup yang dieksekusi dalam segala kondisi, tanpa memandang apakah terjadi exception ataupun tidak.

Secara konvensi, blok ini didedikasikan untuk operasi pelepasan sumber daya (resource cleanup), seperti terminasi koneksi basis data.

Contoh try — catch — finally

```
try {
    System.out.print("Masukkan nilai pembagi: ");
    int angka = Integer.parseInt(sc.nextLine());

    // Alur normal
    System.out.println("Hasil komputasi: " + (100 / angka));

} catch (NumberFormatException e) {
    // Penanganan exception parsing data
    System.out.println("Kesalahan: Input harus berupa bilangan bulat.");

} catch (ArithmeticException e) {
    // Penanganan exception matematika
    System.out.println("Kesalahan: Pembagian dengan nol tidak valid.");

} finally {
    // Dieksekusi pada akhir kondisi apapun
    System.out.println("Siklus operasi selesai dieksekusi.");
    sc.close();
}
```

Penjelasan Eksekusi Exception

Penjelasan Semantik:

Blok try mengisolasi area eksekusi normal. Ketika terjadi anomali, sistem akan melempar (throw) objek exception. Objek tersebut ditangkap (catch) oleh parameter blok penanganan yang berkesesuaian. Setelah evaluasi usai, blok finally dieksekusi sebagai rutinitas akhir.

Alternatif Pendelegasian Penanganan

Apabila sebuah metode tidak diinstruksikan untuk menangani exception secara mandiri, pendelegasian ke metode pemanggil dapat dilakukan menggunakan deklarasi `throws`.

Kata Kunci `throws` & Exception Propagation

Keyword `throws`

Mekanisme untuk menandai sebuah method akan melempar exception, dan memberi tahu pemanggil:

"Tolong implementasikan `try-catch` pada program yang memanggil method ini, atau definisikan ulang `throws` di method yang memanggil."

Exception Propagation

Mekanisme di mana exception yang tidak langsung ditangani dilempar secara beruntun hingga berakhir di tempat yang dikhususkan untuk menanganinya.

```
// Menandai method ini bisa melempar IOException
public static String bacaFile(String path)
    throws IOException {
    return Files.readString(Path.of(path));
}

// Pemanggil WAJIB menangani exception
public static void main(String[] args) {
    try {
        String isi = bacaFile("data.txt");
        System.out.println(isi);
    } catch (IOException e) {
        System.out.println("Gagal: " + e.getMessage());
    }
}
```

Ilustrasi Propagation: 1. Data Layer

1. Repository (Throw)



2. Service (Propagate)



3. Controller (Catch)

```
class UserRepository {  
    public User findById(UUID uid) throws Exception {  
        User searchedUser = null;  
        for (User user: users) {  
            if (user.getUid().equals(uid)) {  
                searchedUser = user;  
                break;  
            }  
        }  
  
        if (searchedUser == null) {  
            // 1. Exception tercipta dan dilempar dari level terbawah  
            throw new Exception("User not found");  
        }  
  
        return searchedUser;  
    }  
}
```

Ilustrasi Propagation: 2. Service Layer

1. Repository (Throw)



2. Service (Propagate)



3. Controller (Catch)

```
class UserService {  
  
    // 2. Exception tidak ditangani di sini, melainkan  
    // didelegasikan (dipropagasi) ke atas menggunakan 'throws'  
    public User findById(UUID id) throws Exception {  
        return userRepository.findById(id);  
    }  
  
}
```

Ilustrasi Propagation: 3. Controller Layer

1. Repository (Throw)



2. Service (Propagate)



3. Controller (Catch)

```
class UserController {  
  
    public Map<String, User> getUserById(UUID id) {  
        Map<String, User> dto = new HashMap<>();  
  
        // 3. Exception akhirnya ditangkap dan ditangani,  
        // mencegah aplikasi mengalami crash.  
        try {  
            User user = userService.findById(id);  
            dto.put("userData", user);  
        } catch (Exception e) {  
            System.err.println("Error: " + e.getMessage());  
        }  
  
        return dto;  
    }  
}
```

Custom Exception

Membuat Exception Sesuai Kebutuhan

Implementasi Custom Exception

Meskipun Java menyediakan beragam class exception bawaan, pada arsitektur perangkat lunak tingkat lanjut, pengembang sering kali perlu merancang hierarki exception yang berorientasi pada konteks domain spesifik (domain-specific exceptions).

Studi Kasus: Pengembangan REST API

Dalam pengembangan antarmuka pemrograman aplikasi (API), entitas exception khusus sangat berguna untuk menyertakan Kode Status HTTP dan pesan respons terstruktur:

- Entitas sumber daya tidak ditemukan
- Kegagalan validasi atas masukan klien
- Penolakan akses akibat kurangnya otorisasi

Metode Implementasi

Pembuatan class exception kustom dilakukan dengan mengimplementasikan konsep pewarisan (inheritance) dari hirarki

`RuntimeException` (untuk unchecked) atau `Exception` (untuk checked).

Struktur Kode Custom Exception

Contoh pembuatan exception spesifik untuk menangani kasus entitas data tidak ditemukan pada aplikasi.

```
// Melakukan pewarisan dari basis RuntimeException
public class DataNotFoundException extends RuntimeException {

    private final int statusCode;

    // Konstruktor menerima pesan error dan menetapkan kode status
    public DataNotFoundException(String message) {
        super(message);
        this.statusCode = 404; // Mengatur HTTP Status Code untuk kondisi Not Found
    }

    public int getStatusCode() {
        return statusCode;
    }
}
```

Custom Exception Pada Kasus REST API

Skenario Kegagalan	Class Exception	Kode HTTP	Deskripsi Respons
Sumber daya tidak ditemukan	<code>DataNotFoundException</code>	404	Data referensi tidak tersedia
Parameter masukan keliru	<code>BadRequestException</code>	400	Format masukan melanggar skema
Otoritas akses ditolak	<code>ForbiddenException</code>	403	Akses ditolak oleh sistem otorisasi

Praktik Terbaik Arsitektural

Pemanfaatan class exception kustom yang terstandardisasi memfasilitasi sentralisasi penanganan error (error handling centralization), sehingga format respons layanan (API) lebih deterministik dan mempermudah integrasi sistem pihak klien.

Integrasi Custom Exception pada Kode

Penerapan custom exception pada tingkat layanan (service) atau logika bisnis aplikasi.

```
// Contoh integrasi pada logic layer
public User retrieveUserRecord(Long userId) {
    // Mencari entitas pengguna ke basis data
    User user = userRepository.findById(userId);

    // Melempar custom exception jika entitas tidak tersedia
    if (user == null) {
        throw new DataNotFoundException(
            "Identitas pengguna tidak teregistrasi pada sistem."
        );
    }

    return user;
}
```

Latihan Minggu 7

Praktikkan Exception Handling!

Latihan — Mesin ATM Sederhana

Buatlah program Java yang mensimulasikan mesin ATM sederhana dengan ketentuan:

1. Custom Exception

Buat dua custom exception extending `RuntimeException`:

— `InsufficientBalanceException`

— `InvalidAmountException`

2. Class ATM

Method `withdraw(double amount)` — gunakan `throw` untuk melempar exception sesuai kondisi.

3. Program Utama dengan try-catch

Tangkap setiap exception dengan pesan yang sesuai. Gunakan `finally` untuk mencetak saldo akhir.

CONTOH OUTPUT:

≡ MESIN ATM ≡

Saldo awal: Rp 1.000.000

Percobaan tarik: Rp 500.000

Penarikan berhasil.

Saldo akhir: Rp 500.000

Percobaan tarik: Rp 800.000

Error: Saldo tidak mencukupi!

Saldo akhir: Rp 500.000

Percobaan tarik: Rp -100

Error: Nominal tidak valid!

Saldo akhir: Rp 500.000

Assignment!

Seperti biasa setiap selesai praktikum pasti akan ada tugas

Assignment

Sistem Validasi Data Mahasiswa

1. Implementasi Custom Exception (Minimal 2 Class)

Terapkan konsep pewarisan terhadap `RuntimeException`. Setiap kelas diwajibkan memuat atribut `statusCode` dan pesan penjelasan. Contoh usulan implementasi:

```
InvalidNimException, InvalidIpkException
```

2. Enkapsulasi & Validasi Atribut pada Class Mahasiswa

Metode mutator (setter) diharuskan memuat mekanisme validasi masukan. Terapkan deklarasi `throw` untuk melontarkan exception spesifik ketika validasi tidak terpenuhi. Ketentuan aturan: Panjang NIM identik dengan 18 karakter numerik, rentang toleransi IPK antara 0.0 hingga 4.0.

3. Implementasi Antarmuka CLI & Eksekusi Try-Catch

Rancang program antarmuka baris perintah (CLI) interaktif untuk memfasilitasi instansiasi entitas mahasiswa. Gunakan blok `try-catch` guna menampung exception yang dideklarasikan, dan tampilkan indikasi kegagalan meliputi kode status dan pesan kesalahan representatif.

REFERENSI EKSPEKTASI OUTPUT:

```
≡≡≡ MODUL INSTANSIASI DATA MAHASISWA ≡≡≡
```

```
Masukkan NIM: 12345
```

```
Kondisi Kegagalan [400]: Panjang NIM tidak sesua
```

```
Masukkan NIM: 140810240074
```

```
Masukkan Nama: Haris Herdiansyah
```

```
Masukkan IPK: 5.0
```

```
Kondisi Kegagalan [400]: Besaran IPK melampaui r
```

```
Masukkan IPK: 3.75
```

```
[INFO]: Referensi entitas mahasiswa berhasil dia
```

Terima Kasih!

Apakah Anda memiliki pertanyaan? Silakan sampaikan pertanyaan dan berdiskusi melalui kanal masing-masing pada platform Discord.

Semoga materi pembelajaran hari ini dapat dipahami dan bermanfaat.